武汉大学

本科毕业论文（设计）

基于共享内存事件格式的嵌入式二进制日志系统设计与实现

姓　　名：邓泽宇

学　　号：2022302111098

专　　业：软件工程

学　　院：计算机学院

指导教师：武小平

二〇二六年三月

原创性声明

本人郑重声明：所呈交的论文（设计），是本人在指导教师的指导下，严格按照学校和学院有关规定完成的。除文中已经标明引用的内容外，本论文（设计）不包含任何其他个人或集体已发表及撰写的研究成果。对本论文（设计）做出贡献的个人和集体，均已在文中以明确方式标明。本人承诺在论文（设计）工作过程中没有伪造数据等行为。若在本论文（设计）中有侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

作者签名：_____　　指导教师签名：_____

日　期：_____

版权使用授权书

本人完全了解武汉大学有权保留并向有关部门或机构送交本论文（设计）的复印件和电子版，允许本论文（设计）被查阅和借阅。本人授权武汉大学将本论文的全部或部分内容编入有关数据库进行检索和传播，可以采用影印、缩印或扫描等复制手段保存和汇编本论文（设计）。

作者签名：_____　　指导教师签名：_____

日　期：_____

摘　要

面向嵌入式与边缘设备，日志系统需要在可观测性、写入开销与并发一致性之间取得平衡。针对文本日志在高频场景下格式化成本高、空间冗余大以及多进程共享元数据易发生初始化竞争的问题，本文提出并实现 Optbinlog。该系统采用事件格式描述文件、共享元数据与二进制编码路径，将格式解析从热路径迁出，并通过偏移访问保证跨进程映射一致性。

本文围绕“机制分析—理论推导—分层实验验证”展开：首先调研 Linux ftrace、LTTng、Zephyr logging、systemd-journald、NanoLog、RT-Thread ulog 与 OpenHarmony HiLog Lite 等代表系统，并定位其关键优化路径；随后建立 N（日志条数）与 D（并发设备/节点数）驱动下的时间、空间与吞吐模型；最后在统一统计口径下完成工程近似层与严格公平层的对照验证。

实验数据由四个阶段构成：工程近似矩阵、严格语义对齐主套件（single、本地多设备 $D = 5, 10, 20, 50$）、真实多设备硬件 CRC32C 复测（5/10/15/20 节点）以及初始化竞争验证。结果表明：binary 相对 text 在本轮纳入语义组上均保持显著正收益；在“Optbinlog 对齐 peer 语义、peer 保持原生存储路径”的严格公平口径下，Optbinlog 在 single 场景已可在时间与吞吐上超过 nanolog/zephyr，但空间仍落后；在真实多设备场景中，对 ulog/syslog 保持稳定综合优势，对 hilog 呈现“时间小幅占优、空间占优、吞吐近似持平”的混合形态，对 nanolog/zephyr 仍处负区；ftrace 则呈现“空间正收益、时间吞吐负收益”。新增空间扫描进一步显示，ulog/syslog 在 $N \approx 10$ 即出现空间反超，hilog 在 $N \approx 50$ 左右出现交叉，而 nanolog/zephyr 在 $N \leq 100000$ 仍未出现交叉点。初始化竞争实验稳定满足“单创建者、其余进程/节点复用已初始化共享文件”的正确性约束。理论趋势与实测结果总体一致。

关键词：嵌入式日志；二进制编码；共享内存；并发初始化；性能建模

ABSTRACT

For embedded and edge systems, logging must balance observability with overhead (storage, CPU, and concurrent initialization). This thesis implements Optbinlog, a schema-shared binary logging approach for embedded deployment. Event metadata are parsed once, materialized in shared memory, and accessed via offsets, so runtime hot-path cost is reduced and cross-process mapping remains stable.

The work first surveys representative domestic and international logging systems (Linux ftrace, LTTng, Zephyr logging, systemd-journald, NanoLog, RT-Thread ulog, OpenHarmony HiLog Lite) and analyzes their optimization code paths. Then a theory-first model is built to derive how time/space/throughput scale with log count N and device count D. Finally, measured results are compared against the derived trends.

The retained dataset follows a stage-based workflow: engineering-approximate matrix rerun, strict-aligned suite (single and local multi-device scans at $D = 5, 10, 20, 50$), hardware-CRC32C real multi-node rerun (5/10/15/20 nodes), and local/node-level initialization competition. Results show that binary consistently outperforms text across the included semantic groups. Under the native-path fairness setting, where Optbinlog aligns to each peer's semantics while the peer keeps its original storage path, binary already surpasses nanolog/zephyr in single-node time and throughput but still trails them in space; in real multi-node runs it stays clearly ahead of ulog/syslog, remains mixed against hilog, and stays behind nanolog/zephyr on time and throughput. In addition, ftrace in real multi-node runs shows a split pattern (space-positive but time/throughput-negative). The added space-scan experiment shows crossover around $N \approx 10$ for ulog/syslog, around $N \approx 50$ for hilog, while no crossover is observed for nanolog/zephyr up to $N = 100000$. Initialization-race results satisfy the single-creator property consistently. Overall, theoretical trends and empirical observations are aligned.

Keywords: embedded logging; binary encoding; shared metadata; concurrency; performance modeling

目录

摘要 I

ABSTRACT II

1 绪论 1

1.1 研究背景 1

1.2 研究问题 1

1.3 本文工作 1

1.4 研究方法与工作中心 1

1.5 研究创新点 2

1.6 论文结构安排 2

2 国内外研究调查与缺陷分析 2

2.1 调研范围与方法 2

2.2 国际代表系统与优化思路（代码级） 3

2.3 国内生态系统与优化思路（代码级） 4

2.4 Optbinlog 与六类 peer 系统的对比 4

2.5 使用场景与工作中心差异 6

2.6 典型优化代码入口（可追溯） 7

2.7 现有方案缺陷归纳 7

2.8 Optbinlog 对缺陷的对应关系 7

2.9 复现边界与结论适用范围 8

3 Optbinlog 设计迭代与工程实现 8

3.1 设计目标 8

3.2 事件格式与元数据结构 9

3.3 写入/读取路径 9

3.4 关键迭代 10

3.5 与实际场景的匹配性 11

3.6 工程化读取能力补全 11

4 理论模型与数学推导 12

4.1 符号定义 12

4.2 从算法与数据结构到单条代价 12
4.3 空间随 N 的详细推导 16
4.4 时间随 N 的详细推导 17
4.5 多设备 D 的详细推导 19
4.6 模式间理论排序与可检验预测 21
4.7 与第 5 章指标的映射关系 22
4.8 本章小结 22

5 实验迭代、对比与理论一致性分析 23
5.1 实验设计与证据组织 23
5.2 工程近似层与严格公平层的分工 24
5.3 单机场景：严格对齐下的基线关系 25
5.4 本地多设备：竞争项放大后的趋势 26
5.5 真实多设备：共享格式文件下的节点级结果 29
5.6 日志量扩展：空间交叉点实验 34
5.7 理论—实证对照 36
5.8 原因分析与消融佐证 37
5.9 初始化竞争正确性 38
5.10 证据链组织与外推边界 39
5.11 机制解释与应用启示 39

6 结论与展望 40
6.1 研究结论 40
6.2 当前局限 41
6.3 未来展望 41

参考文献 41

致谢 43
6.4 附录 A 关键复现实验命令（阶段化复现） 43

1 绪论

1.1 研究背景

嵌入式系统日志常处于“高频写入、有限存储、多进程并发、实时约束”并存的环境。传统文本日志在可读性上有优势，但会带来：

1. 字符串格式化与解析开销高。
2. 单条日志冗余字节多，长期空间成本高。
3. 多进程共享格式信息时容易出现初始化竞争和一致性问题。

因此，本文并不局限于 text 与 binary 的局部比较，而是将问题置于“理论分析、实现机制与分组对照实验”统一框架下进行讨论。

1.2 研究问题

1. 现有日志系统的主流优化点是什么？是否仍有缺口？
2. Optbinlog 的实现路径能否覆盖这些缺口？
3. 从算法与数据结构出发，N 与 D 的时间/空间/吞吐趋势应如何推导？
4. 推导结论与实验结果哪些一致，哪些不一致，原因是什么？

1.3 本文工作

1. 完成国内外日志系统调研，并定位代表性优化代码入口。
2. 设计并实现 Optbinlog（事件格式解析、共享元数据、二进制读写、并发初始化状态机）。
3. 建立抽象数学模型，推导不同模式在 N、D 维度下的性能趋势。
4. 组织多轮实验（单机高负载、多设备模拟、真实多设备模拟（多台虚拟机节点）并发、init-race）并进行理论对照分析。

1.4 研究方法与工作中心

围绕前述研究问题，本文将工作中心放在三条主线上：1. **机制主线**：从日志系统算法与数据结构出发，不先下结论，先解释为什么会快/慢、为什么会大/小。2. **对照主线**：不只做 text 对比，还和 syslog/ftrace/nanolog/zephyr/ulog/hilog 做同

语义分组对照。3. **验证主线**：统一统计口径与可复现实验脚本，按“理论推导 → 单机 → 多设备 → 真实多设备模拟（多台虚拟机节点）”逐层验证。

1.5 研究创新点

1. 提出“共享 schema + 偏移访问”的用户态二进制日志实现，重点解决多进程初始化一致性与重复解析问题。
2. 从算法与数据结构角度推导 N 与 D 对时间、空间和吞吐的影响，并给出可检验交叉点与饱和点表达式。
3. 建立“工程近似层 + 严格公平层”两级实验体系：前者保留真实系统路径差异，后者统一 schema、统一生成值与统一平台，避免把语义体量差或输入来源差误判为实现优劣。
4. 在真实多设备模拟（多台虚拟机节点）场景下验证初始化竞争与并发写入行为，形成可复现实验流程。

1.6 论文结构安排

全文按“问题提出—方案设计—理论建模—实验验证—结论展望”组织：1. 第 2 章给出国内外代表系统的代码级调研与缺陷归纳，明确 Optbinlog 的问题边界。2. 第 3 章说明 Optbinlog 的关键设计、工程实现与迭代路径。3. 第 4 章建立 N 与 D 驱动下的时间、空间与吞吐模型，并给出可检验预测。4. 第 5 章按单机、本地多设备、真实多设备模拟与日志量增长扫描四类实验进行验证，并完成理论对照与原因分析。5. 第 6 章总结结论、说明局限并给出后续优化计划。

---

2 国内外研究调查与缺陷分析

2.1 调研范围与方法

本章基于官方文档、论文与代码仓库进行调研，聚焦“热路径优化机制”和“可复现实现入口”。为保证后续章节的横向可比性，调研统一按四个维度展开：数据结构组织、关键算法路径、并发行为特征与工程部署边界。

调研对象如下：1. Linux ftrace / kernel ring buffer。2. LTTng。3. Zephyr logging subsystem。4. systemd-journald。5. NanoLog（USENIX ATC 2018）。6. RT-Thread

ulog。7. OpenHarmony HiLog Lite。

2.2 国际代表系统与优化思路（代码级）

2.2.1 Linux ftrace（内核追踪）

代码与文档显示其核心是 per-CPU lockless ring buffer。在 ring_buffer.c 中，写入路径围绕 “预留 → 填充 → 提交” 组织，关键是把竞争局部化到 CPU 本地缓冲，降低全局锁竞争。

1. 数据结构：每 CPU buffer + page 链式组织，维护 head/commit/tail。
2. 算法逻辑：优先本地写入、批量可见性提交，读写阶段解耦。
3. 场景定位：内核 tracing、高并发诊断；功能强但接入复杂度高于应用层日志库。

2.2.2 LTTng（Linux Tracing Toolkit Next Generation）

LTTng 采用 tracepoint instrumentation + lockless buffer 管线，强调高吞吐追踪和离线分析。

1. 数据结构：channel/sub-buffer 等追踪缓冲结构。
2. 算法逻辑：将事件采集与后处理分层，追踪点开销可控。
3. 场景定位：系统级性能分析与可观测平台，部署和运维要求较高。

2.2.3 Zephyr logging（RTOS 场景）

在 Zephyr Kconfig.mode 与 log_msg.c 可见，关键机制是 deferred logging。

1. 数据结构：log_msg 对象 + 消息池（MPSC packet buffer）。
2. 算法逻辑：前台线程只做封包入队，后台线程统一输出；支持 overflow/drop 策略。
3. 场景定位：RTOS 中实时任务优先，要求前台日志路径尽量短、可控。

2.2.4 systemd-journald

其 JOURNAL_FILE_FORMAT 与 journald-server 路径强调二进制对象与索引。

1. 数据结构：header + objects + 哈希/索引。
2. 算法逻辑：写入与检索并重，保证一致性与查询能力。
3. 场景定位：通用 Linux 系统服务日志；对轻量嵌入式而言依赖偏重。

2.2.5 NanoLog（研究型高性能日志）

从 NanoLogCpp17.h、RuntimeLogger.cc 可见，NanoLog 核心是“编译期提取格式 + 运行期仅记录动态参数”。

1. 数据结构：staging buffer、runtime logger 队列/缓冲。
2. 算法逻辑：前台路径尽量只做参数打包，压缩与格式恢复转移到后台/离线。
3. 场景定位：超高频日志、允许离线恢复文本的性能敏感场景。

2.3 国内生态系统与优化思路（代码级）

2.3.1 RT-Thread ulog

ulog.c 提供异步输出和多 backend 抽象，思路是“前台快写缓冲，后台慢速输出”。

1. 数据结构：ringbuffer/rbb、backend 列表。
2. 算法逻辑：异步线程统一调度输出，减轻业务线程阻塞。
3. 场景定位：RT-Thread 设备侧调试与在线日志。

2.3.2 OpenHarmony HiLog Lite

HiLog Lite 在格式能力上做约束（减少复杂格式），换取更低资源开销。

1. 数据结构：轻量日志缓冲和简化格式存储。
2. 算法逻辑：限制格式复杂度，降低格式化路径成本。
3. 场景定位：资源受限设备的系统日志基础设施。

2.4 Optbinlog 与六类 peer 系统的对比

表 2.1 Optbinlog 与六类 peer 系统对比

系统	核心机制	典型使用场景	主要优势	主要代价	与 Optbinlog 的关系
Linux ftrace	内核 tracing ring buffer	内核/驱动诊断、时序追踪	内核路径短、追踪能力强	接入复杂、与业务日志模型存在语义差异	Optbinlog 适合业务日志结构化记录，Linux ftrace 更适合底层追踪
Zephyr logging	deferred 入队 + 后台批刷	RTOS 实时任务日志	前台阻塞小	后台线程调度与队列竞争	工程复现中以 native-like deferred 路径逼近其原生机制；Optbinlog 不依赖后台线程，行为更确定
systemd-journald	文本日志 + 系统日志守护进程	通用 Linux 服务运维	生态成熟、检索与管理工具丰富	文本格式化与系统路径开销较高	Optbinlog 在高频写入场景更强调紧凑编码与可控开销
NanoLog	固定字段二进制直写	超高频、可离线恢复文本	调用点开销极低	语义扩展与工具链耦合强	工程复现中以 native-like 固定字段二进制路径逼近其原生机制；Optbinlog 可扩展性更强，但热路径常数项更高

（续表 2.1）

系统	核心机制	典型使用场景	主要优势	主要代价	与 Optbinlog 的关系
RT-Thread ulog	异步队列 + 多 backend	嵌入式设备在线调试	工程集成灵活	后台文本格式化与后端竞争	工程复现中以 native-like 异步队列路径逼近其原生机制；Optbinlog 在结构化与一致性方面更强
OpenHarmony HiLog Lite	受限字段 + 轻量头	资源受限系统日志	资源占用较低	语义表达能力受限	工程复现中以 native-like 轻量头路径逼近其原生机制；Optbinlog 提供更完整字段语义和校验能力
Optbinlog	共享 schema + typed 编码 + CRC	多进程边缘网关/控制器	跨进程一致性、结构化可扩展、空间可控	初始化和通用编码固定成本	作为“语义完整 + 工程可迁移”的折中方案

2.5 使用场景与工作中心差异

如表2.1所示，Optbinlog 在设计上更偏向“语义完整、跨进程一致和工程可迁移”的折中路线，因此与追求极限热路径或系统级 tracing 的方案形成了明显分工。

1. 若目标是**内核态追踪**，Linux ftrace 仍是优先选择；若目标是**应用层结构化日志**，Optbinlog 更匹配。
2. 若目标是**极限热路径时间**，NanoLog/Zephyr logging 这类强调短热路径的方案更有优势；若目标是**语义可扩展与跨进程一致性**，Optbinlog 更稳定。

3. 若系统强调运维生态兼容，systemd-journald 优势明显；若系统强调嵌入式长期在线与空间效率，Optbinlog 更具针对性。

2.6 典型优化代码入口（可追溯）

表 2.2 典型优化代码入口与定位

系统	关键优化点	代码/文档入口
Linux ftrace	每 CPU 无锁 ring buffer	ring_buffer.c，设计文档 ring-buffer-design
LTTng	tracepoint + lockless buffer	LTTng Concepts 官方文档
Zephyr logging	deferred + 消息池	Kconfig.mode、log_msg.c
systemd-journald	二进制对象与索引文件	journald-server.c、JOURNAL_FILE_FORMAT
NanoLog	编译期格式抽取 + 运行时二进制参数	NanoLogCpp17.h、RuntimeLogger.cc
RT-Thread ulog	异步输出与多 backend	ulog.c + ulog 文档
OpenHarmony HiLog Lite	受限格式降低开销	hiviewdfx_hilog_lite 文档/代码

2.7 现有方案缺陷归纳

从嵌入式角度可抽象出四类常见缺口：

1. **格式化开销仍在热路径**：text/syslog 在高频场景下受字符串格式化与内核接口调用影响明显。
2. **元数据复用不足**：多进程系统若每进程重复解析格式，启动和内存成本上升。
3. **并发初始化一致性风险**：共享资源初次创建缺少状态机时，容易出现竞态。
4. **跨平台复现实验缺失**：只在单平台得到的结论容易失真，外推边界较弱。

2.8 Optbinlog 对缺陷的对应关系

1. 用 “事件格式文件 + 二进制编码” 替代热路径字符串格式化。

2. 用共享元数据文件持久化 tag/element 描述，进程间偏移复用。
3. 用 INITIALIZING → INITIALIZED 状态机控制并发初始化。
4. 用统一脚本输出 mean/median/p95/std/95%CI 与 IQR 过滤结果，支持多轮迭代对比。

2.9 复现边界与结论适用范围

本文按照“代码阅读先行、实验复现跟进”的顺序开展验证工作：

1. 已完成官方代码级阅读：ftrace、Zephyr、NanoLog、ulog 等核心实现入口。
2. 受网络与平台限制：无法完整拉取/编译全部外部系统并在本机跑 Linux tracing 全栈。
3. 已执行算法级复现：实现 nanolog_like、zephyr_deferred_like、ulog_async_like、hilog_lite_like 及“语义对齐”版本（第 5 章给出数据）。

上述约束已在第 5 章实验方法中显式记录，用于区分“工程可复现边界”与“算法层结论边界”。因此，第 5 章中的排序结论均在对应口径下成立，不对未覆盖平台作超范围外推。

---

3 Optbinlog 设计迭代与工程实现

3.1 设计目标

1. 降低单条日志热路径开销（时间）。
2. 降低长期写入空间消耗（空间）。
3. 支持多进程共享元数据与安全初始化（并发）。
4. 对齐嵌入式可部署需求（工程可落地）。

上述目标在后续章节中分别对应为可测指标：时间目标映射到 elapsed/end_to_end，空间目标映射到 total_bytes，并发目标映射到初始化竞争正确性与节点级稳定性，工程目标映射到跨场景复现与部署可迁移性。

3.2 事件格式与元数据结构

当前 Optbinlog 采用 “共享元数据文件 + 进程内共享视图缓存 + 自描述二进制帧 + CRC32C(hw) + string-heavy 场景自动 varstr” 的实现方式。

事件定义采用 (name|type|bits) 形式，字段类型简化为 L/D/S：

1. L：定长整数，按 schema 指定字节数写入。
2. *D*：双精度浮点，固定写入 8 字节。
3. S：字符串字段；当前实现同时支持固定宽度字符串编码和 varstr 编码。

共享元数据文件内仍由以下结构组织：

1. OptbinlogSharedTag：共享文件头与初始化状态。
2. OptbinlogBitmap：tag 合法性位图。
3. OptbinlogEventTag：事件定义入口。
4. OptbinlogEventTagEle：元素类型与长度描述。

在共享文件之上，系统进一步引入了**进程内共享视图缓存层**。该缓存按共享文件的 dev/inode/size/mtime 判定是否可复用，并缓存：

1. 已映射的共享文件视图。
2. 每个 tag 的 OptbinlogTagCacheEntry。
3. schema 级统计量，例如 tag 数、总 payload 字节数、字符串字段总宽度、字符串字段数和最大字符串字段宽度。

这些统计量会直接参与写路径中的 varstr 自动决策，因此它们属于当前热路径优化的一部分。

3.3 写入/读取路径

写入与读取路径遵循 “先判定元数据状态、再执行编码/解码、最后进行一致性校验” 的统一流程。

写入路径：1. 通过 acquire_shared_view() 获取共享文件视图；若共享文件未变化，则直接复用进程内缓存的 tag cache。2. 根据 schema 统计量判断是否启用 varstr；当字符串字段占比高或最大定长字符串字段较大时，默认切换到 varstr。3. 依据 tag cache 顺序编码 payload。4. 构造 4 字节自描述帧头，其中同时编码

payload_len、varstr 标志位和校验类型。5. 对单条记录计算校验值；默认采用 CRC32C(hw)，仅在硬件不可用时回退到软件实现。6. 将 frame_header + payload + checksum 写入用户态缓冲区，按批次落盘。

读取路径：1. 读取 4 字节帧头，并解析 payload_len、varstr 标志位与 checksum_type。2. 读取 payload 与校验字段，并按帧头声明的校验类型执行校验。3. 按共享格式解码：整数和浮点按固定宽度读取，字符串则根据 varstr 标志选择 “长度前缀 + 数据” 或 “固定宽度截断/补零” 方式。4. 将解码后的记录交给统一回调接口。

因此，当前记录格式可写作：

$$\text{frame} = \text{frame_header} \parallel \text{payload} \parallel \text{checksum}$$

其中：

$$\text{frame_header} = \text{payload_len} \cup \text{varstr_bit} \cup \text{checksum_type}$$

3.4 关键迭代

1. 从 “每事件独立结构” 迭代为 “统一结构 + 位图 + 偏移表”。
2. 从进程内指针共享迭代为 “偏移量共享”（解决跨进程映射基址差异）。
3. 从 switch-case 查找迭代为 arr=id/100, idx=id%100 的常数级路径。
4. 从软件 CRC32 迭代为默认 CRC32C(hw)；在 ARMv8 与 SSE4.2 平台优先使用硬件指令，软件实现只作为回退路径。
5. 新增自描述帧头，使 CRC32 / CRC32C / none 与 fixed-string / varstr 可以在同一读取逻辑中兼容。
6. 新增进程内共享视图缓存和 tag cache，把共享文件 reopen、tag 遍历与 schema 统计从稳态热路径迁出。
7. 新增 schema 级自动 varstr 策略：当字符串字段占比高时，从固定宽度字符串编码切换到 “长度前缀 + 实际长度” 编码。
8. 增加并发初始化状态机和 init-race 压力验证，保证共享文件只被单创建者初始化，其余进程或节点复用既有元数据。

综合以上迭代，当前实现可归纳为三条关键收敛点：

1. **校验语义与热路径代价兼顾**：系统保留每条记录校验，同时通过

CRC32C(hw) 将校验成本压缩到更适合热路径的水平。

2. **共享元数据的稳态收益进入真实路径**：共享视图缓存和 tag cache 使 schema 解析与 tag 遍历不再重复进入每次写入。
3. **字符串主导 schema 的冗余写入被抑制**：在 string-heavy 场景下自动启用 varstr，把固定宽度补零和无效拷贝降到更低。

3.5 与实际场景的匹配性

1. 小样本 N：共享文件打开、共享视图校验、文件打开和缓冲准备等固定项占比仍然较高，时间优势不一定立刻出现。
2. 中大样本 N：固定项被摊薄，自描述帧头和 CRC32C(hw) 的额外代价收敛到常数级，binary 的时间和空间优势更稳定。
3. string-heavy schema：自动 varstr 会显著减少补零写入和冗余字节，因此空间优势更容易显现。
4. 高并发 D：即使校验和 schema 解析成本被压低，调度、文件竞争和写出路径仍会主导吞吐峰值与回落点。
5. 多进程/多节点部署：共享 schema、进程内缓存复用和节点级初始化竞争正确性已经形成闭环，因此该设计更贴近真实工程部署。

因此，第 4 章将把上述实现行为进一步抽象为 $A_m/C_m/B_m/W_m$ 等参数形式，便于与第 5 章实验结果逐项对照。

3.6 工程化读取能力补全

为提升实验数据复核效率，本文在写入与解码库之外补充了 optbinlog_read 工具，用于“共享格式文件 + 二进制日志文件”的直接可视化读取。该工具具备三项能力：1. 按 tag 与字段名输出记录，避免只看到匿名值序列。2. 支持 table/jsonl 两种格式，便于命令行排错与自动化分析脚本复用。3. 支持 --limit 与 --summary，可快速截取样本并统计各 tag 分布。

该工具把“论文中的统计结果”与“底层二进制记录”建立了可追溯链路，使实验结论不只停留在汇总图表层，而能够直接回溯到具体记录级证据。

4 理论模型与数学推导

本章先做抽象推导，不代入具体事件文件数值。为保证推导可解释且可验证，本文采用以下前提：在单次统计窗口内事件分布近似稳定；各模式在同一场景下使用一致的输入语义；模型参数以期望意义解释而非单点极值解释。

4.1 符号定义

1. N：总日志条数。
2. D：并发设备数（或并发进程数）。
3. r：每设备日志条数，满足 $N = D \cdot r$。
4. k：事件类型（tag）索引，$k \in \{1, \dots, T\}$。
5. p_k：事件类型 k 的概率，满足 $\sum_k p_k = 1$。
6. e_k：事件 k 的字段数。
7. $l_{k,j}$：事件 k 第 j 个字段编码字节数。
8. $L_k = \sum_{j=1}^{e_k} l_{k,j}$：事件 k 的有效载荷字节数。
9. A_m：模式 m 的固定时间开销（初始化、打开文件、预热）。
10. C_m：模式 m 的单条边际时间开销。
11. B_m：模式 m 的单条边际空间开销。
12. $S_{0,m}$：模式 m 的固定空间开销（例如共享元数据）。
13. Q_m：吞吐（records/s）。

除特别说明外，本章的“空间”均对应实验统计中的 total_bytes：对 binary 为“日志文件 + 共享格式文件且共享文件只计一次”，对其他模式为其实际输出文件大小；“时间”默认对应单轮写入阶段的均值趋势，若切换到 end_to_end 口径，则把进程/节点启动与调度准备吸收到固定项中。

4.2 从算法与数据结构到单条代价

为避免“只看结果不看机制”，先把每种模式拆成操作级成本。以下表达式均以单条记录期望开销为核心，再扩展到总量关系。

4.2.1 text

单条记录包含：1. 读取输入字段。2. snprintf/拼接格式串。3. 写入用户态缓冲并最终落盘。

时间可写为：

$$t_{\text{text}}(k) = t_{\text{parse}}(k) + t_{\text{fmt}}(k) + t_{\text{write}}(\text{bytes}_{\text{text}}(k))$$

空间可写为：

$$b_{\text{text}}(k) = b_{\text{prefix}} + b_{\text{delim}} + b_{\text{num}}(k) + b_{\text{str}}(k)$$

其中 $b_{\text{num}}(k)$ 是数字转十进制后的字符长度，随数值大小缓慢变化。

4.2.2 binary（optbinlog）

当前 optbinlog_binlog_write() 的稳态路径可分为四步：1. 复用共享视图与 tag cache，并按 tag_id 完成常数级查找。2. 按字段类型编码 payload。3. 根据校验模式计算记录校验。4. 将记录写入用户态批量缓冲区，缓冲区满后统一写出。

与代码一致，单条记录的物理布局为：

$$\text{frame} = \text{frame_header}_{4B} \parallel \text{payload} \parallel \text{checksum}_{4B}$$

其中 payload 并不只是 schema 字段本身，还额外包含：

$$\text{payload} = \text{timestamp}_{8B} \parallel \text{tag_id}_{2B} \parallel \text{ele_count}_{1B} \parallel \text{fields}$$

记记录内部固定元数据开销为：

$$g_{\text{bin}} = 8 + 2 + 1 = 11$$

若字符串字段按固定宽度写出，则 payload 中 schema 字段的有效载荷仍记为 L_k；若 schema 触发 varstr，则对应地记为 $\tilde{L}_k$。于是单条时间可写为：

$$t_{\text{bin}}(k) = t_{\text{lookup}} + t_{\text{encode}}(k) + t_{\text{checksum}}(k) + \bar{t}_{\text{write}}(b_{\text{bin}}(k))$$

其中 $\bar{t}_{\text{write}}(\cdot)$ 表示 “经用户态缓冲摊薄后的平均写出代价”，比逐条系统调用

更接近当前实现。

单条空间则可写为：

$$b_{\text{bin}}(k) = 4 + g_{\text{bin}} + \tilde{L}_k + 4$$

固定空间只在共享格式文件层面体现，因此有：

$$S_{0,\text{bin}} = S_{\text{shared}}$$

若 schema 触发 varstr，则字符串字段的有效载荷不再按固定宽度写出，而是写为“长度前缀 + 实际长度”：

$$\tilde{L}_k = \sum_{j \in \mathcal{N}_k} l_{k,j} + \sum_{j \in \mathcal{S}_k} \left(2 + \hat{l}_{k,j}\right), \qquad \hat{l}_{k,j} \le l_{k,j}$$

还需说明两点与代码实现直接对应的口径：1. 默认 CRC32C(hw) 覆盖的是“帧头 + payload”；兼容模式下 CRC32 覆盖 payload，none 则不做校验。2. 共享文件的打开、映射与 tag cache 构建只在视图失效时重新执行，因此它们主要进入固定项 A_{bin}，而不是每条记录的边际项。

4.2.3 syslog / ftrace

抽象为“系统日志接口 + 内核路径”：

$$t_{\text{sys}}(k) = t_{\text{format_sys}}(k) + t_{\text{syscall}} + t_{\text{kernel_path}} + t_{\text{write_kernel}}$$

$$t_{\text{ftrace}}(k) = t_{\text{format_ftrace}}(k) + t_{\text{trace_write}} + t_{\text{ring_or_file_path}}$$

空间：

$$b_{\text{sys}}(k) = b_{\text{sys_header}} + b_{\text{sys_payload}}(k)$$

$$b_{\text{ftrace}}(k) = b_{\text{ftrace_header}} + b_{\text{ftrace_payload}}(k)$$

4.2.4 nanolog_like / zephyr_deferred_like / ulog_async_like / hilog_lite_like

nanolog_like（固定字段二进制）:

$$t_{\text{nano}} = t_{\text{pack_fixed}} + t_{\text{memcpy}} + t_{\text{write}}$$

$$b_{\text{nano}} = \text{const}$$

zephyr_deferred_like（前台入队 + 后台批刷）:

$$t_{z,\text{front}} = t_{\text{enqueue}} + t_{\text{signal}}$$

$$t_{z,\text{back}} = \frac{t_{\text{dequeue}} + t_{\text{batch_write}}(k_b)}{k_b}$$

$$t_z = t_{z,\text{front}} + t_{z,\text{back}}$$

$$b_z = b_{\text{queue_node}} + b_{\text{fixed}} + \frac{b_{\text{batch_meta}}}{k_b}$$

ulog_async_like（异步队列 + 文本 backend）:

$$t_{\text{ulog}} = t_{\text{enqueue}} + t_{\text{cond_signal}} + \frac{t_{\text{dequeue}} + t_{\text{format_text}} + t_{\text{backend_write}}}{k_b}$$

$$b_{\text{ulog}} = b_{\text{queue_node}} + b_{\text{text_payload}}$$

hilog_lite_like（受限字段 + 轻量头）:

$$t_{\text{hilog}} = t_{\text{compact_format}} + t_{\text{write}}(h_{\text{lite}} + p_{\text{lite}})$$

$$b_{\text{hilog}} = b_{\text{header_lite}} + b_{\text{payload_lite}}$$

4.3 空间随 N 的详细推导

4.3.1 期望单条字节

对任意模式 m，定义：

$$B_m = \mathbb{E}[b_m(k)] = \sum_k p_k\, b_m(k)$$

则总空间：

$$S_m(N) = S_{0,m} + NB_m \tag{4.1}$$

这是严格的一阶线性模型，斜率为 B_m，截距为 $S_{0,m}$。见公式（4.1）所示，后续空间分析都可以归结为“固定项 + 单条斜率”两部分。

4.3.2 binary 的显式表达

由 4.2.2 得：

$$B_{\text{bin}} = 8 + g_{\text{bin}} + \sum_k p_k \tilde{L}_k$$

$$S_{\text{bin}}(N) = S_{\text{shared}} + N\left(8 + g_{\text{bin}} + \sum_k p_k \tilde{L}_k\right) \tag{4.2}$$

其中，$g_{\text{bin}} = 11$ 对应 payload 内部固定元数据，外层的两个 4B 分别对应帧头和校验字段。见公式（4.2）所示，binary 的总空间同时受“每条记录固定项”和“共享格式文件固定项”影响。

4.3.3 节省率与导数

binary 相对模式 x 的空间节省率：

$$\eta_{\text{bin}/x}(N) = 1 - \frac{S_{\text{bin}}(N)}{S_x(N)}$$

若 $S_{0,x} = 0$，则

$$\eta_{\text{bin}/x}(N) = 1 - \frac{B_{\text{bin}} + S_{\text{shared}}/N}{B_x}$$

求导：

$$\frac{d\eta_{\text{bin}/x}}{dN} = \frac{S_{\text{shared}}}{N^2 B_x} > 0$$

说明 N 越大，S_{shared}/N 越小，binary 空间优势越稳定。这说明只要边际斜率满足优势条件，共享文件这一固定项就会随着 N 增大而被持续摊薄。

4.3.4 空间交叉点

当比较 bin 与 x：

$$S_{\text{bin}}(N) \leq S_x(N)$$

$$S_{\text{shared}} + NB_{\text{bin}} \leq S_{0,x} + NB_x$$

若 $B_x > B_{\text{bin}}$，可得交叉点：

$$N^*_{\text{space}}(\text{bin}, x) = \frac{S_{\text{shared}} - S_{0,x}}{B_x - B_{\text{bin}}} \tag{4.3}$$

当 $N > N^*_{\text{space}}$ 时，binary 空间一定优于 x。因此，空间交叉点的判断可以直接按公式（4.3）所示进行。

4.4 时间随 N 的详细推导

4.4.1 从单条时间到总时间

定义：

$$C_m = \mathbb{E}[t_m(k)] = \sum_k p_k \, t_m(k)$$

则总时间：

$$T_m(N) = A_m + NC_m \tag{4.4}$$

该形式直接对应实验中的 elapsed/write_only/end_to_end 指标。见公式（4.4）所示，总时间同样由固定项和边际项组成。若统计口径切换到 end_to_end，则进程/节点启动、文件准备与调度等待也并入 A_m。

4.4.2 吞吐公式与单调性

$$Q_m(N) = \frac{1000N}{T_m(N)} = \frac{1000}{C_m + A_m/N} \tag{4.5}$$

对 N 求导：

$$\frac{dQ_m}{dN} = \frac{1000A_m}{N^2\left(C_m + A_m/N\right)^2} > 0$$

说明在模型中吞吐随 N 单调上升并渐近：

$$\lim_{N\to\infty} Q_m(N) = \frac{1000}{C_m}$$

在 $A_m > 0$ 与 $C_m > 0$ 条件下，吞吐增速会随 N 增大而下降，表现为“先快后慢”的收益递减，这与实验中大 N 后增益变缓一致。因此，公式（4.5）给出了第 5 章解释“吞吐为何先快后慢趋于平台”的统一表达。

4.4.3 时间交叉点（binary 何时优于其他模式）

比较 bin 与 x：

$$T_{\text{bin}}(N) \le T_x(N)$$

$$A_{\text{bin}} + NC_{\text{bin}} \le A_x + NC_x$$

当 $C_x > C_{\text{bin}}$ 时：

$$N^*_{\text{time}}(\text{bin}, x) = \frac{A_{\text{bin}} - A_x}{C_x - C_{\text{bin}}} \tag{4.6}$$

解释：1. 若 A_{bin} 较大（共享初始化重），小 N 可能不占优。2. 随 N 增大，NC 项主导，只要 $C_{\text{bin}} < C_x$，最终会跨过交叉点。

因此，某一模式何时在时间上完成反超，可按公式（4.6）所示理解。

4.4.4 C_m 的字段复杂度展开

为把“常数项差异”写清楚，可把边际开销展开到字段数级别：

$$C_m = c_{m,0} + c_{m,1}\mathbb{E}[e_k] + c_{m,2}\mathbb{E}[L_k] + c_{m,3}\mathbb{E}[\text{syscall}_m]$$

其中：1. $\mathbb{E}[e_k]$ 决定字段遍历成本（编码循环次数）。2. $\mathbb{E}[L_k]$ 决定 memcpy、校验和写出字节相关成本。3. $\mathbb{E}[\text{syscall}_m]$ 反映模式是否显著依赖系统日志接口或内核路径。

结合当前复现代码，可得到更贴近实现的解释：1. text_semantic_like 主要承担字段格式化与文本展开成本。2. binary 的边际项主要由 typed encode、校验和批量写出摊销组成，固定项则额外包含共享视图与 tag cache 准备。3. nanolog_like / zephyr_deferred_like 走固定宽度打包路径，边际项通常较低，但 zephyr_deferred_like 还会受到批刷组织方式影响。4. ulog_async_like / syslog / ftrace 会显著暴露文本展开、系统接口或内核路径的常数开销。5. hilog_lite_like 通过轻量头和受限消息体压缩了单条记录规模，因此常落在二进制直写组与文本系统组之间。

4.5 多设备 D 的详细推导

4.5.1 总时间分解

设每设备写 r 条：

$$E_m(D, r) = A_{\text{spawn}}(D) + A_m + rC_m + W_m(D, r) \tag{4.7}$$

见公式(4.7)所示，多设备总时间除了单机边际项外，还需要显式计入并发等待项。若观察的是 write_only 指标，则 $A_{\text{spawn}}(D)$ 会明显缩小；若观察 end_to_end，它则不能忽略。

其中 W_m 为竞争等待项（锁、队列、内核资源、I/O 饱和）。

4.5.2 队列化近似（解释吞吐饱和）

把共享瓶颈抽象成单服务台，服务率 μ_m（records/s），到达率 $\lambda = D\nu$（ν 为单设备提交率）。

稳定条件：

$$\rho = \frac{\lambda}{\mu_m} < 1$$

等待时间近似随 ρ 增大而快速上升，可抽象为：

$$W_m(D,r) \sim r f_m(\rho), \qquad f_m'(\rho) > 0, \qquad f_m''(\rho) > 0$$

因此当 D 接近

$$D_{\text{sat},m} \approx \frac{\mu_m}{\nu}$$

吞吐进入平台区；超过后出现明显回落。这里的“服务台”不一定是真实队列，也可以对应共享文件写出通道、后台批刷线程或内核 tracing 接口；该抽象的意义在于用同一符号解释不同模式的饱和与回落。

4.5.3 常用二次近似

在工程拟合中常写作：

$$E_m(D,r) \approx \beta_m + rC_m + \alpha_m D + \gamma_m D^2$$

总吞吐：

$$Q_m(D,r) = \frac{1000Dr}{E_m(D,r)}$$

当 $\gamma_m > 0$ 时，D 过大必然导致分母增长过快，出现峰值后下降。

进一步对 D 求导：

$$\frac{dQ_m}{dD} \propto \left(\beta_m + rC_m + \alpha_m D + \gamma_m D^2\right) - D\left(\alpha_m + 2\gamma_m D\right)$$

整理得：

$$\frac{dQ_m}{dD} \propto \beta_m + rC_m - \gamma_m D^2$$

故吞吐峰值点近似为：

$$D_{\text{peak},m} = \sqrt{\frac{\beta_m + rC_m}{\gamma_m}} \tag{4.8}$$

这给出了可直接与实验扫描结果比对的“理论峰值设备数”。其中，峰值设备数可直接按公式（4.8）与实验扫描结果对照。

4.5.4 空间随 D

$$S_m(D,r) = S_{0,m} + DrB_m$$

因此空间对 D 严格线性，差异主要由斜率 B_m 与截距 $S_{0,m}$ 决定。这一点与当前实验统计口径一致：在记录规模 r 固定时，设备数增加只会线性放大记录总量，而不会改变每条记录的平均编码体积。

4.6 模式间理论排序与可检验预测

4.6.1 异语义比较（最短路径口径）

在工程近似层中，各模式的输入语义和存储组织并不完全一致，因此本节只讨论“机制上更可能出现的方向”，而不把它写成固定全序。

从当前复现代码出发，可得到三条较稳定的预期：1. 固定宽度二进制直写组（如 nanolog_like、zephyr_deferred_like）通常拥有更低的单位记录空间与更短的边际写入路径。2. 文本展开或系统路径更重的模式（如 text/syslog/ftrace/ulog_async_like）通常拥有更高的常数项。3. binary 与 hilog_lite_like 处于二者之间：前者保留共享 schema、自描述帧和校验语义，后者保留轻量头和受限消息体，因此二者的排序会更依赖具体 schema 与统计口径。

可检验预测：1. nanolog_like 在时间与吞吐上常呈数量级优势。2. hilog_lite_like 在“受限语义”下应明显优于 text/syslog。3. ulog_async_like 在吞吐未饱和且后台轻负载时有机会受益；若后台文本后端成为瓶颈，可能慢于 binary/text。4. 上述优势不代表语义等价下仍保持同倍率。

4.6.2 语义对齐比较（公平口径）

本文所说的“语义对齐”，并不是把所有 peer 强行改写成同一种记录格式，而是在**同一组字段语义、同一组字段值与同一平台**下，让 Optbinlog 与 peer 分别走各自的 native-like 复现路径。于是：

1. binary 的差异主要来自共享 schema、frame 组织、per-record CRC 以及字段遍历。

2. peer 的差异主要来自其 native-like 记录布局、批刷/后台路径、文本展开与系统接口路径。

因此，公平口径下的空间关系不再由“语义体量不同”主导，而主要由各模式的单位记录常数项主导。对于 nanolog_like / zephyr_deferred_like 这类紧凑二进制 peer，B_{peer} 往往会长期小于 B_{bin}；对于 ulog_async_like / hilog_lite_like / syslog 这类仍带有消息头、文本展开或系统路径附加开销的 peer，B_{bin} 则更有可能在较小 N 后越过交叉点。

C_{peer} 与 C_{bin} 的差异同样主要来自实现常数，包括编码循环、缓存命中、批刷组织、格式化路径与校验实现，而不再主要来自“消息语义更多或更少”的偏差。

可检验预测：1. 时间差异将由各模式的原生热路径决定，而不是由语义字段数决定。2. 空间差异会在同一语义组内呈现稳定斜率，并在 N 增长时暴露出是否存在交叉点。

4.7 与第 5 章指标的映射关系

为了让推导可被实验验证，建立公式到指标的映射：

1. $T_m(N)$ 对应 elapsed_ms 或 end_to_end_ms 的均值趋势。
2. Q_m 对应 throughput_rps / throughput_e2e_rps。
3. S_m 对应 total_bytes。
4. N^*_{time} 与 N^*_{space} 对应“某场景下 binary 由劣转优”的阈值。
5. $D_{\text{sat},m}$ 对应“多设备扫描中吞吐峰值后回落”的拐点。

本章给出的是“由算法与数据结构直接推出的理论趋势”；第 5 章再代入实测数据检验这些趋势是否成立，以及偏离原因。

4.8 本章小结

本章完成了三项工作：一是把不同日志模式统一到可比较的时间与空间参数形式；二是给出 N 与 D 两个核心自变量下的交叉点、峰值点与线性斜率含义；三是建立理论参数与实验指标的映射关系。基于这些结果，第 5 章将按“单机 → 本地多设备 → 真实多设备模拟 → 日志量扫描”的顺序进行逐层验证。

5 实验迭代、对比与理论一致性分析

5.1 实验设计与证据组织

本章按“实验口径定义 → 分场景结果 → 理论对照 → 原因解释”的顺序展开，避免将语义体量差异误判为实现差异。实验主线分为两层：工程近似层回答“真实系统路径代价如何排序”；严格公平层回答“同语义、同平台、同统计口径下 Optbinlog 与 peer 的本征差异”，统一以 text_semantic_like / binary / peer native mode 为比较对象，覆盖以下三类场景：1. 单机高负载（single）。2. 本地多设备模拟（$D = 5, 10, 20, 50$ 扫描）。3. 真实多设备模拟（多台虚拟机节点）并发（5/10/15/20 节点，每节点一台设备）。

此外，本文同步更新了两组初始化竞争结果：本地多进程初始化运行和节点级初始化运行。统计口径统一为 mean/median/p95/std/95%CI，先执行 IQR（1.5）异常值过滤，再输出时间、空间、吞吐及改进百分比，以保证各组实验可横向比较。

实验数据按用途分成两层。grouped_semantic_matrix 承担“工程近似对照”角色，用于观察系统级路径代价；严格语义对齐数据覆盖 single、local multi-device（$D = 5, 10, 20, 50$）和真实多设备节点扫描（5/10/15/20 节点）的硬件 CRC32C 复测。本轮严格对齐口径已将 syslog 纳入 single/multi/l1 三层对比，并在 l1 场景额外纳入 ftrace。其核心原则是：**Optbinlog 根据各 peer 的日志语义设计对应的 eventlogst_semantic_*，所有模式共享同一组字段值生成器，而 peer 侧保持原生存储路径**。针对“日志量增加后空间交叉点何时出现”的问题，本文额外加入 $N = 1 \sim 100000$ 的空间扫描实验。消融结果只用于解释原因，不承担主结论角色。

为保证分析口径可追溯，本章额外固定“对比对象、运行平台与数据量”三类约束。尤其在真实多设备（多台虚拟机节点）阶段，本轮采用“**全节点共享同一 shared_eventtag.bin 格式缓存文件**、节点结果文件独立输出”的方式，重点观察共享元数据在节点级并发中的竞争与收益释放。

维度	本文实验口径
对比对象	text_semantic_like（文本基线）、binary（Optbinlog 最终路径）；single/multi peer 包含 syslog/nanolog/zephyr/ulog/hilog，l1 peer 包含 sys-log/ftrace/nanolog/zephyr/ulog/hilog，且 peer 侧保持原生存储路径
单机/本地多设备平台	macOS 26.3（Darwin 25.3.0，ARM64）
真实多设备平台	Lima Ubuntu 24.04 ARM64 虚拟机集群；最多 20 节点；节点 CPU 为 2 vCPU（aarch64）
校验实现与硬件路径	binary 默认每条记录 CRC32C(hw)；L1 节点 aarch64 CPU flags 包含 crc32，基准按 -march=armv8-a+crc 编译
真实多设备数据量	每节点 records=20000；节点规模 5/10/15/20；repeats=3，warmup=1
真实多设备总样本	六个语义组 × 三种模式 × 节点扫描后，有效统计样本约 54,000,000 条日志（不含 warmup）
真实多设备总日志字节	有效轮次聚合约 4.601 GiB（按 cluster_total_bytes 汇总）
真实多设备空间口径	集群总空间；共享格式文档在每个模式下仅计算一次
空间交叉点附加扫描	records 从 1 到 100000 共 13 个点；repeats=2；指标为 total_bytes

5.2 工程近似层与严格公平层的分工

本轮更新后，syslog 已进入严格公平层三类场景，ftrace 已进入严格公平层真实多设备场景。工程近似层仍然保留其独立价值：用于观察“系统路径全链路”的工程代价；严格公平层用于回答“同语义、同平台条件下”的实现差异。两层

数据并行报告，有助于避免把系统路径差异误判为编码机制差异。

5.3 单机场景：严格对齐下的基线关系

本节结果来自严格语义对齐套件的合并汇总文件。三类模式统一采用同一 eventlogst_semantic_* 语义组、同一生成器产生的字段值，以及同一平台；其中 binary 采用 “共享 schema/tag cache + per-record CRC32C(hw) + string-heavy auto-varstr” 的当前实现，而 peer 侧保持各自原生存储路径。

表 5.1 单机高负载下 binary 相对 text 的结果

分组	binary 相对 text 时间%	binary 相对 text 空间%	binary 相对 text 吞吐%	结论
syslog	+71.79%	+45.75%	+254.99%	相对文本路径收益明显
nanolog	+92.12%	+69.79%	+1202.37%	相对文本路径收益明显
zephyr	+93.01%	+69.52%	+1333.26%	相对文本路径收益明显
ulog	+92.81%	+68.96%	+1291.05%	相对文本路径收益明显
hilog	+92.92%	+71.95%	+1323.50%	相对文本路径收益明显

表 5.2 单机高负载下 binary 相对 peer 的结果

分组	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	结论
syslog	+71.33%	+45.75%	+249.52%	严格语义对齐后，binary 在三项指标上均明显领先
nanolog	+8.63%	-41.76%	+12.58%	时间与吞吐已转正，但空间仍显著落后
zephyr	+26.18%	-41.76%	+35.12%	单机热路径已转正，空间仍明显吃亏
ulog	+82.02%	+59.43%	+456.18%	binary 明显优于文本扩展更重的 peer 路径
hilog	+74.53%	+32.61%	+295.22%	binary 在三项指标上总体占优

如表5.1和表5.2所示，单机场景呈现出更清晰的 “双层分化”。第一，binary vs text_semantic_like 在五组上全部为正，说明只要把字段值固定下来，文本格

式化与字符串拼接就会稳定抬高 text 路径成本。第二，切换到 peer 原生路径后，binary 对 nanolog/zephyr 的时间与吞吐已转入正区，但空间仍稳定落在负区；对 ulog/hilog/syslog 则在三项指标上整体保持正收益。也就是说，在单机稳态下，Optbinlog 已经能够在执行时间上压过紧凑二进制 peer，但每条记录的 frame/CRC 常数项仍使其在空间上难以反超 nanolog/zephyr。

因此，工程近似层与严格公平层需要分别承担不同问题：前者回答“真实系统路径大致如何排序”，后者回答“在同一 schema、同一字段值、同一平台下，Optbinlog 相对 peer 的本征开销是什么”。

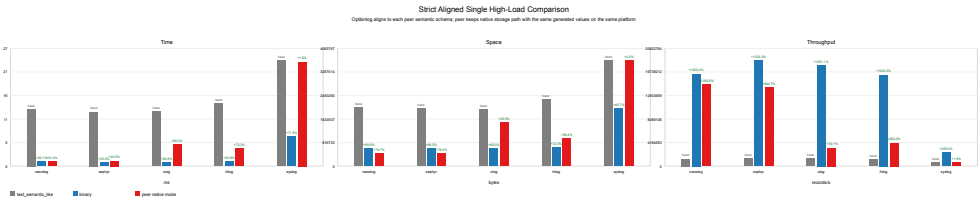

图 **5.1** 严格语义对齐单机高负载总览图

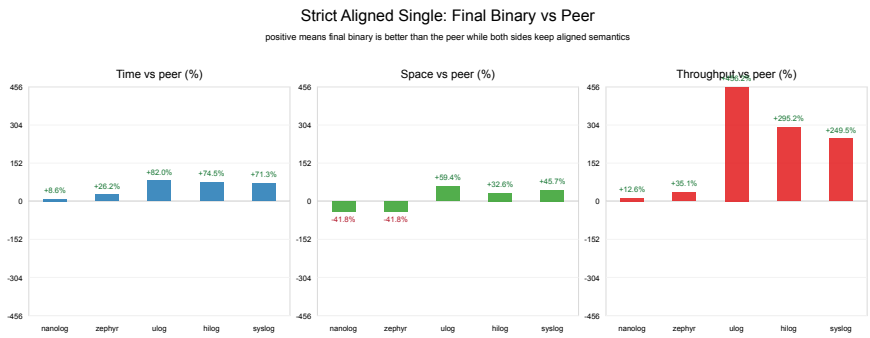

图 **5.2** 严格语义对齐单机场景中 binary 相对 peer 的差值图

5.4 本地多设备：竞争项放大后的趋势

本节结果同样来自 final_aligned_summary.json，但切换到 multi 部分，并对 $D = 5, 10, 20, 50$ 进行扫描。该场景保持平台不变，只让设备数增加，因此能够单独观察 $W_m(D, r)$ 对趋势的影响。

表 **5.3** 本地多设备模拟下 binary 相对 peer 的扫描结果

分组	设备数 D	binary 相对 peer 时间%	binary 相对 peer 吞吐%	binary 相对 peer 空间%	现象
syslog	5	+9.28%	+11.25%	+45.68%	低设备数已明显领先

（续表 5.3）

分组	设备数 D	binary 相对 peer 时间%	binary 相对 peer 吞吐%	binary 相对 peer 空间%	现象
syslog	10	+13.76%	+15.55%	+45.72%	中等并发保持稳定正区
syslog	20	+10.31%	+11.68%	+45.73%	中高并发仍保持正收益
syslog	50	-8.60%	-3.61%	+45.74%	高设备数下时间吞吐回落，但空间仍保持优势
nanolog	5	-5.23%	-2.24%	-42.12%	时间吞吐接近持平，但空间持续为负
nanolog	10	-0.38%	+0.62%	-41.90%	中等并发接近零附近波动
nanolog	20	-7.69%	-7.17%	-41.78%	竞争项放大后再次转负
nanolog	50	+0.96%	+0.50%	-41.71%	时间吞吐小幅回正，空间仍无改观
zephyr	5	-4.57%	-3.40%	-42.14%	低设备数下略处负区
zephyr	10	+7.25%	+6.89%	-41.90%	中等并发阶段性转正
zephyr	20	+0.39%	+0.40%	-41.78%	接近持平，但空间仍为负
zephyr	50	-12.77%	-8.55%	-41.71%	高设备数下再次转负
ulog	5	-58.91%	-23.30%	+59.32%	低设备数下异步路径启动波动明显
ulog	10	+3.00%	+3.19%	+59.39%	中等并发开始转入正区
ulog	20	+32.31%	+24.78%	+59.42%	中高并发优势最明显
ulog	50	+3.15%	+3.17%	+59.44%	高设备数仍保持正区
hilog	5	+3.78%	+3.65%	+32.41%	低设备数小幅领先
hilog	10	-8.18%	-8.68%	+32.53%	中等并发回落到负区
hilog	20	+2.67%	+2.85%	+32.59%	中高并发再次转正
hilog	50	-11.56%	-7.60%	+32.63%	高设备数下时间吞吐再次波动

如表5.3所示，本地多设备场景最显著的变化，**是时间/吞吐开始围绕零点上下波动，而空间差异几乎保持不变**。nanolog/zephyr 的空间百分比始终稳定在约 -42%，ulog/hilog/syslog 则分别稳定在约 +59%、+33% 与 +46%；这说明在固定语义组下，设备数 D 主要改变的是调度竞争与共享路径等待项，而不是单位记录的空间斜率。与之对应，时间和吞吐不再呈单向趋势，而是随不同 peer 的后台刷新、短热路径和共享文件访问方式出现振荡。

这一结果与第 4 章模型中的 $W_m(D,r)$ 完全对应：本地多设备阶段最先被放大的，不是编码体量本身，而是调度竞争和共享路径抖动。因此，D 的增大并不会自动把 binary 推向统一的“更优”或“更差”，而是会让不同 peer 原生路径的等待项被分别放大出来。

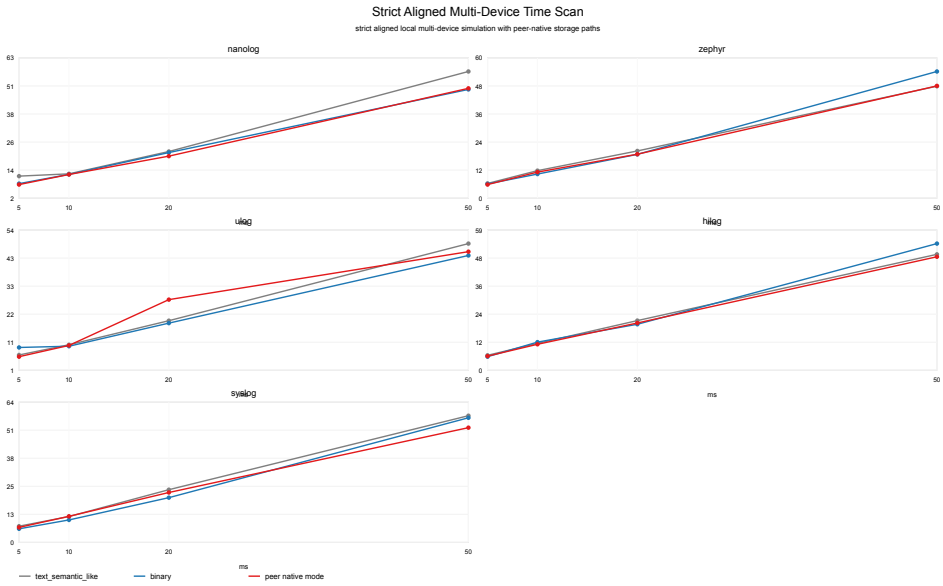

图 **5.3** 严格语义对齐本地多设备时间扫描图

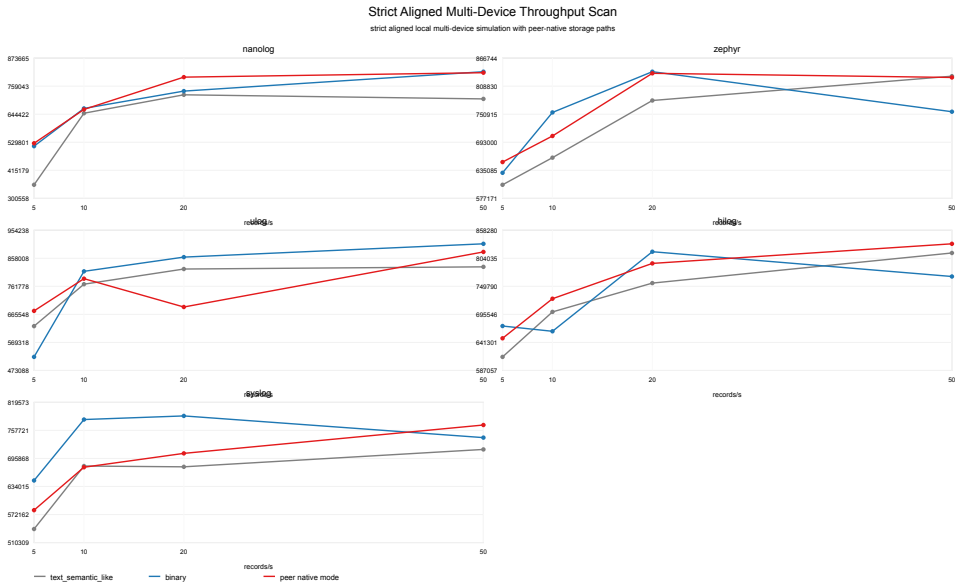

图 **5.4** 严格语义对齐本地多设备吞吐扫描图

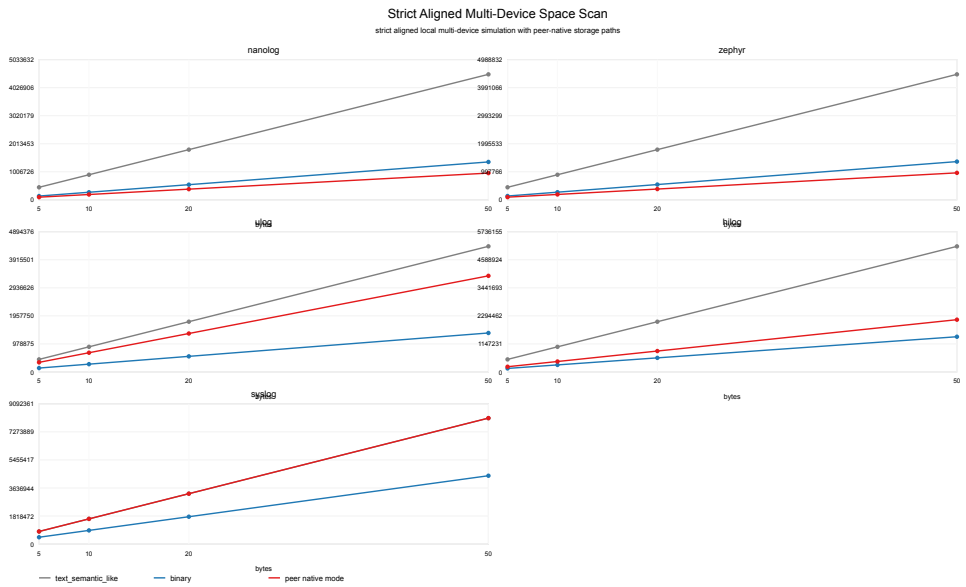

图 **5.5** 严格语义对齐本地多设备空间扫描图

5.5 真实多设备：共享格式文件下的节点级结果

本节结果来自严格对齐套件的 11 节点结果及其合并汇总，采用 5/10/15/20 个并发节点、每节点一台设备的扫描方式。与本地多设备不同，这一场景在保持语义完全对齐的同时，引入了节点级冷启动、VM 调度噪声和跨节点执行偏移，因此更接近论文中的“工程部署代理场景”。本轮真实多设备复测统一使用硬件 CRC32C（aarch64 + -march=armv8-a+crc），并额外约束“全节点共享同一 shared_eventtag.bin 格式缓存文件”，用于验证共享元数据在节点级并发下的趋势变化。为便于与前文表格保持一致，表5.4先给出 20 节点最大规模的 binary vs text 截面，表5.5再给出 binary vs peer 的节点扫描结果。

表 5.4 真实多设备模拟下 binary 相对 text 的结果

分组	节点有效数	binary 相对 text 时间%	binary 相对 text 空间%	binary 相对 text 吞吐%
syslog	20/20	+51.64%	+45.76%	+107.77%
ftrace	20/20	+41.48%	+56.12%	+69.57%
nanolog	20/20	+37.57%	+69.81%	+89.41%
zephyr	20/20	+53.28%	+69.54%	+138.04%
ulog	20/20	+40.51%	+68.98%	+95.41%
hilog	20/20	+54.75%	+71.97%	+149.92%

表 5.5 真实多设备模拟下 binary 相对 peer 的结果

分组	节点数	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	现象
syslog	5	+64.16%	+45.76%	+158.72%	三指标均为正，且优势明显
syslog	10	+64.80%	+45.76%	+161.15%	中节点规模下保持稳定领先
syslog	15	+74.85%	+45.76%	+229.29%	竞争增大时优势继续扩大
syslog	20	+64.54%	+45.76%	+166.10%	20 节点下仍为稳定正收益
ftrace	5	-208.24%	+64.95%	-65.81%	tracing 路径下时间与吞吐显著落后，但空间更优
ftrace	10	-102.05%	+64.95%	-59.53%	高并发下仍保持 “空间优、时间劣” 特征
ftrace	15	-193.94%	+64.95%	-66.79%	结果方向稳定，说明模式差异具有结构性
ftrace	20	-162.95%	+64.95%	-64.07%	20 节点下该特征仍然维持
nanolog	5	-187.09%	-41.68%	-67.22%	紧凑二进制 peer 在节点级并发下明显更快更小
nanolog	10	-393.81%	-41.68%	-77.16%	10 节点处负区最深，竞争项被显著放大
nanolog	15	-262.53%	-41.67%	-72.22%	虽有回升，但仍处深负区
nanolog	20	-274.04%	-41.67%	-72.93%	20 节点下依旧难以接近交叉点
zephyr	5	-148.73%	-41.69%	-56.56%	deferred/batch 原生路径在节点级并发下更优
zephyr	10	-56.38%	-41.68%	-41.54%	10 节点阶段性回升，但仍未转正
zephyr	15	-124.43%	-41.67%	-56.41%	节点继续增加后再次回落
zephyr	20	-99.79%	-41.67%	-51.92%	20 节点仍保持显著负区
ulog	5	+10.27%	+59.45%	+10.46%	低节点数下已整体转正
ulog	10	+30.06%	+59.45%	+61.80%	中节点规模优势最明显
ulog	15	+40.44%	+59.45%	+38.78%	节点增加后仍保持稳定正区
ulog	20	+19.51%	+59.45%	+28.72%	20 节点仍具清晰综合优势
hilog	5	-6.04%	+32.65%	-6.17%	低节点数下时间吞吐略落后，但空间占优
hilog	10	+5.78%	+32.66%	-0.98%	时间转正、吞吐接近持平

（续表 5.5）

分组	节点数	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	现象
hilog	15	+17.00%	+32.66%	-0.05%	中高节点数下时间优势扩大
hilog	20	+6.67%	+32.66%	-7.39%	20 节点仍呈“时间小优、吞吐近持平、空间占优”

如表5.4和表5.5所示，真实多设备模拟给出了比本地多设备更稳定的分组结果。首先，20 节点最大规模下 binary vs text_semantic_like 在六组上全部保持明显正收益，说明一旦引入节点级并发与系统路径，文本格式化仍然是稳定的高开销基线。其次，ulog/syslog 在 5/10/15/20 四个点位上都保持对 peer 的稳定综合正收益；hilog 则呈现更细的混合结构，即时间小幅占优、空间持续占优、吞吐接近持平或小幅为负。再次，nanolog/zephyr 在“全节点共享同一格式缓存文件”口径下始终保持明显负区；ftrace 仍呈现“空间正收益、时间和吞吐显著负收益”的稳定分化。这表明节点级并发与共享文件竞争会把不同 peer 原生路径的结构性差异进一步放大。

因此，“真实多设备下谁更优”并不是单机结论的简单复制，而是对第 4 章模型中 $W_m(D,r)$ 与冷路径项 A_m 的联合验证：ulog/syslog 这类文本扩展和系统路径开销更重的 peer，在三指标上更容易被 binary 持续压过；hilog 处在中间地带，表现为“空间更优、时间略优、吞吐接近持平”；nanolog/zephyr 则因原生路径足够紧凑，在该口径下仍未越过交叉点；ftrace 展示的则是“空间更紧凑但时延更高”的 tracing 路径特征。

需要说明的是，本节表格里的空间指标统一采用“集群总空间且共享格式文档只计一次”口径，因此不会再出现“共享文档按节点重复累计”的偏差；在该口径下，空间绝对值仍随节点数近似线性增长，与第 4 章线性空间模型一致。

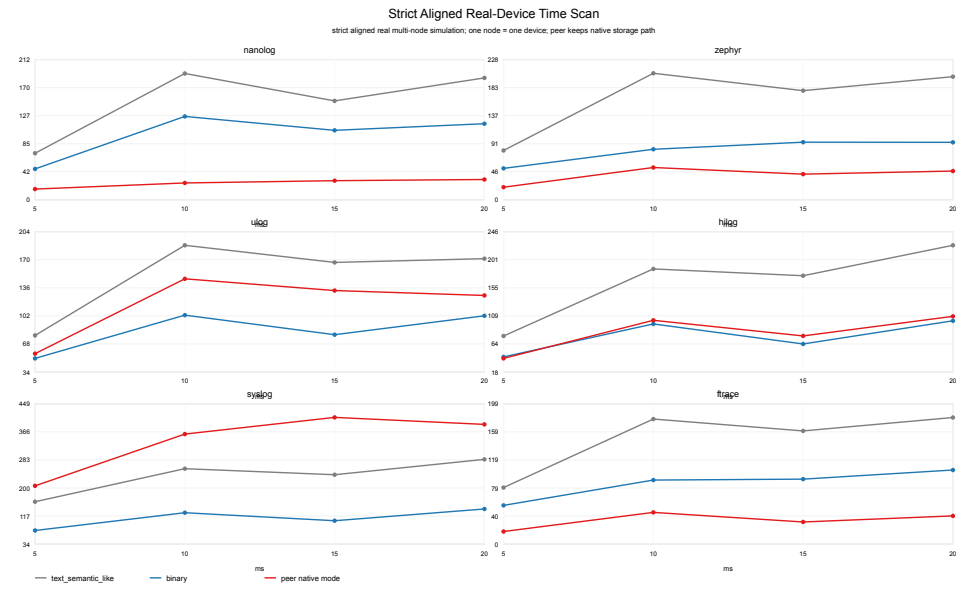

图 5.6 严格语义对齐真实多设备时间扫描图

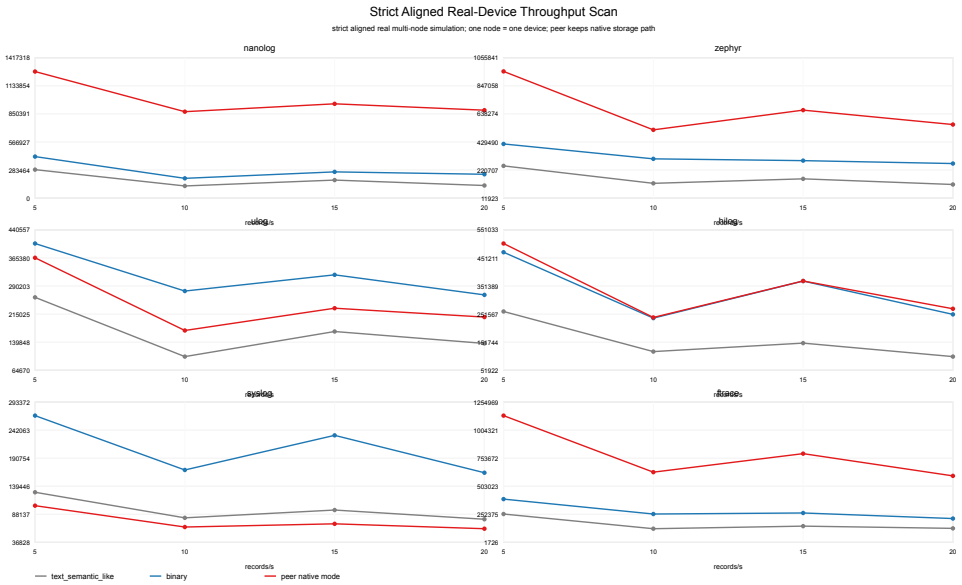

图 5.7 严格语义对齐真实多设备吞吐扫描图

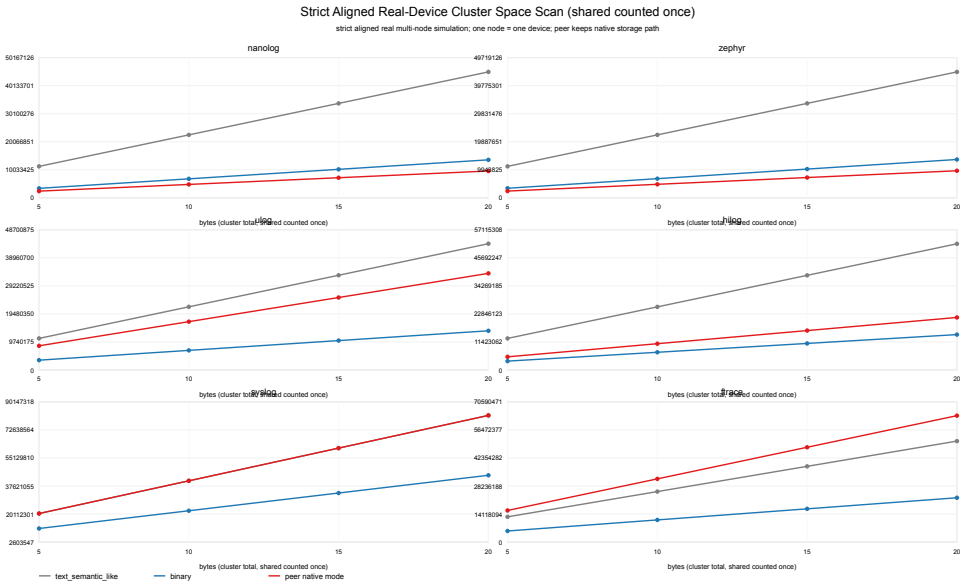

图 **5.8** 严格语义对齐真实多设备集群总空间扫描图

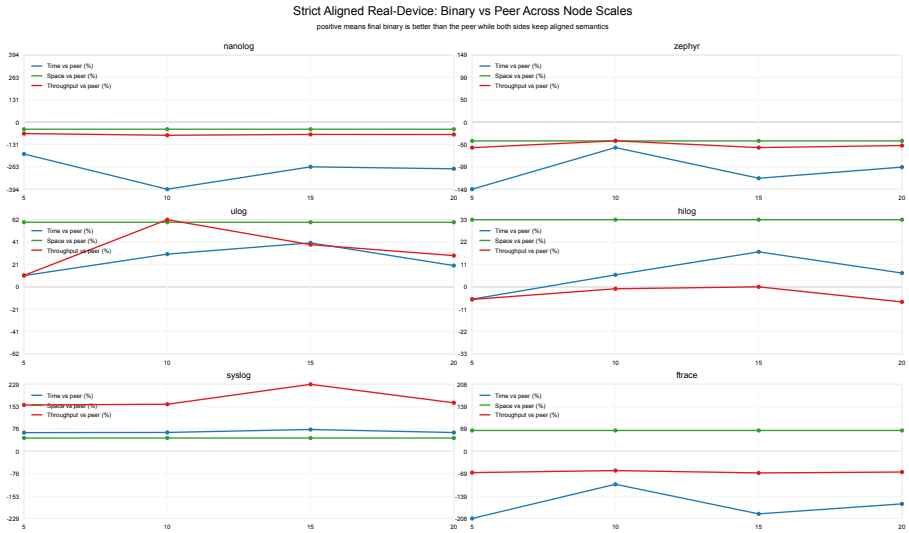

图 **5.9** 严格语义对齐真实多设备场景中 binary 相对 peer 的节点扫描差值图

图5.6、图5.7 与图5.9 共同支持 “ulog/syslog 在时间-吞吐维度整体为正、hilog 处于时间略优且吞吐近持平的中间地带、zephyr/nanolog 持续负区、ftrace 呈空间正收益但时间吞吐负收益” 的主结论；图5.8 则给出空间口径的直接证据：当统计对象改为 “集群总空间” 后，各组曲线均随节点数近似线性增长，与第 4 章线性空间模型一致。

5.6 日志量扩展：空间交叉点实验

为了回答“日志量增加到什么程度，Optbinlog 的空间性能会优于其他模式”这一问题，本文在严格语义对齐口径下新增 N 扫描实验：单设备单进程，模式固定为 text_semantic_like / binary / peer native mode，记录数扫描点为 1,2,5,10,20,50,100,200,500,1000,5000,20000,100000，每点 repeats=2、warmup=0，统计指标为 total_bytes 均值。交叉点定义为“最早满足 binary_bytes ≤ peer_bytes 的 N”。

表 5.6 日志数据量扫描下 binary 相对 peer 的空间交叉点结果

分组	经验交叉点 N^*	线性拟合交叉点	趋势解读
syslog	10	6.0	$N \approx 10$ 出现交叉，随后 binary 空间持续更优
nanolog	未出现（至 100000）	-43.9	在正 N 区间内不出现交叉，binary 空间持续大于 peer
zephyr	未出现（至 100000）	-45.2	在正 N 区间内不出现交叉，binary 空间持续大于 peer
ulog	10	9.7	$N \approx 10$ 即出现交叉，之后 binary 空间持续更优
hilog	50	29.7	$N \approx 30 \sim 50$ 后才出现交叉，说明轻量头路径仍保留一定空间优势

如表5.6、图5.10与图5.11所示，空间维度仍然呈现清晰分组，但交叉点位置比旧版结论更细化。ulog/syslog 在较小日志量（约 $N = 10$）即越过交叉点，说明文

本展开或系统路径带来的增量字节会很快累积；hilog 则要到 $N \approx 30 \sim 50$ 左右才完成反超，表明其轻量头路径在小日志量下仍保留一定空间优势；nanolog/zephyr 两组在扫描上限 $N = 100000$ 仍未出现交叉，说明其 peer 原生路径的单位记录空间效率在当前 schema 下持续优于 binary。该实验给出了更可操作的工程结论：若目标场景主要对标 ulog/syslog 风格且日志量达到 $N \geq 10$，或对标 hilog 风格且日志量达到 $N \geq 50$，Optbinlog 在空间上可预期优于 peer；若对标 nanolog/zephyr 风格，则应优先优化 binary 的单位记录编码开销，再讨论交叉点外推。

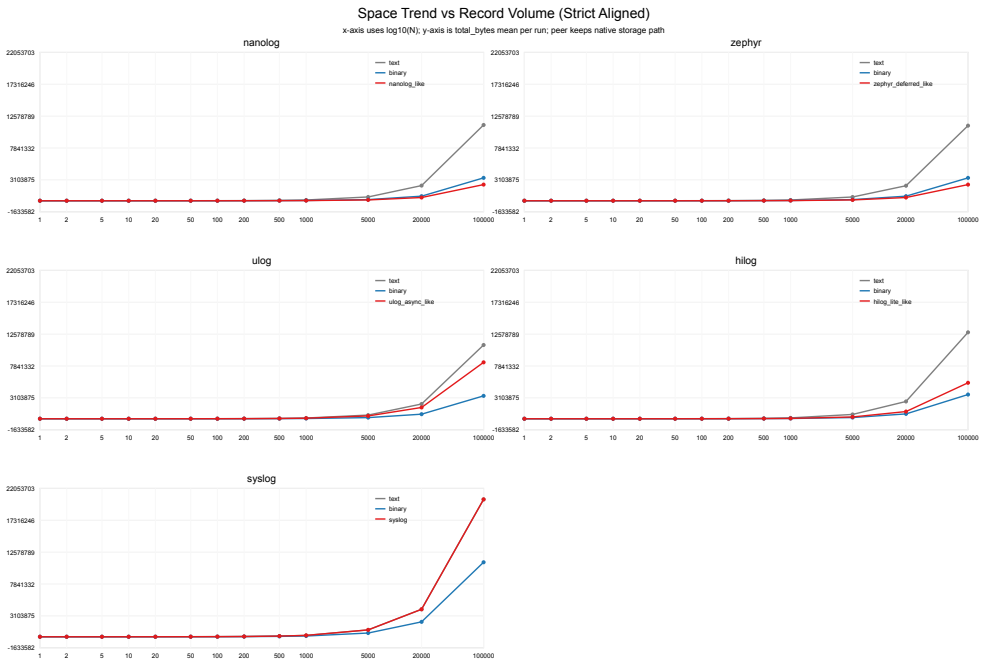

图 **5.10** 日志数据量扫描下各模式空间曲线图

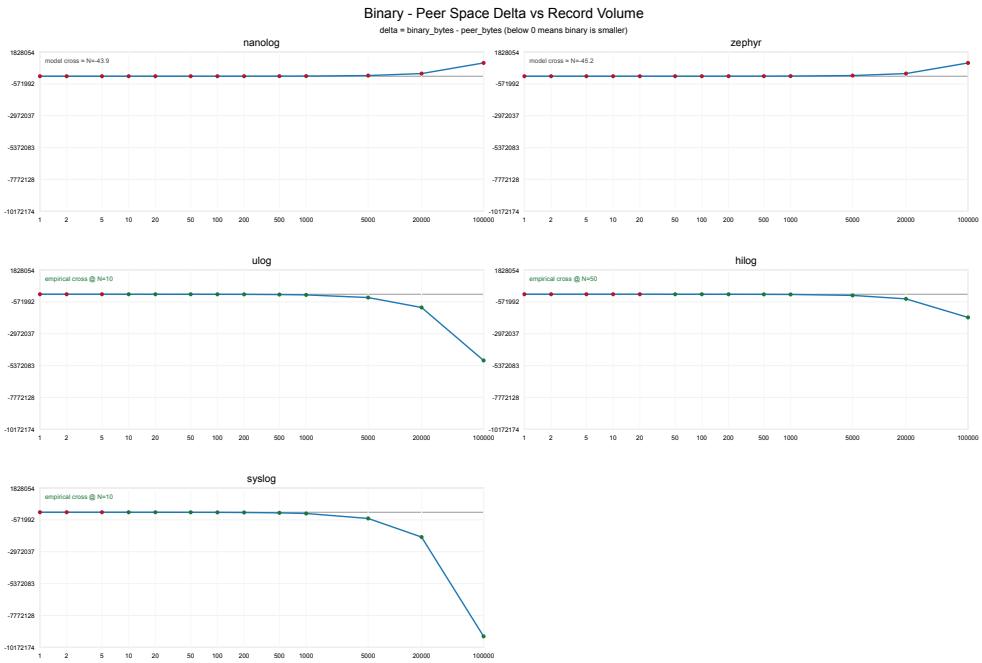

图 **5.11** 日志数据量扫描下 binary 相对 peer 的空间差值图

5.7 理论—实证对照

综合 5.2 ~ 5.5 与第 4 章模型，可得到五点结论：

1. 空间模型与实测一致：total_bytes 随 N 线性增长，不同语义组对应不同 B_m。在 nanolog/zephyr 组中，binary 相对 peer 的空间始终为负；在 ulog/hilog/syslog 组中，binary 的空间长期为正；在 l1 的 ftrace 组中，binary 空间同样保持稳定正收益。
2. 时间与吞吐的交叉点是“按语义组和平台共同出现”的，而不是“按模式统一出现”的。nanolog/zephyr 在 single 可转正，但到 l1 又重新落入深负区；ulog/syslog 在 l1 呈现稳定正收益；hilog 则位于二者之间，更接近“时间略优、吞吐近持平”的中间形态；ftrace 在 l1 时间维度持续深负。
3. 竞争项 $W_m(D, r)$ 在本地多设备与真实多设备场景中都可被直接观察到。本地多设备主要表现为“时间/吞吐围绕零点振荡而空间斜率稳定”，真实多设备则进一步把这种波动放大为“持续正区/持续负区/近似持平”的结构性分化。
4. “真实多设备”不是简单复制“本地多设备”。L1 场景额外引入了节点级冷启动、VM 调度噪声和每节点新进程启动，因此 A_m 的占比会明显抬升；这正是 nanolog/zephyr 在 L1 重新转入深负、ulog/syslog 在三指标上仍可保持清晰正收益、hilog 只保留部分优势，以及 ftrace 呈现“空间优先但时延劣

势” 的主要原因。

5. 日志量增长实验进一步给出了空间交叉点阈值：ulog/syslog 在 $N \approx 10$ 附近即可完成 binary ≤ peer 的空间反超，hilog 的反超点后移到 $N \approx 30 \sim 50$，而 nanolog/zephyr 在 $N \leq 100000$ 仍未出现交叉点，说明 “空间何时反超” 同样是语义组相关结论。

5.8 原因分析与消融佐证

为了更直接解释上述三层场景的排序差异，本文继续使用严格公平消融运行作为 “原因分析层”，但不再把它作为主体结论数据集。该数据集给出的关键消融结果如下：

表 5.7 消融实验中的冷路径与校验影响

分组	binary 冷路径 prep(ms)	binary_nocrc 时间改善%
syslog	8.214	+2.45%
nanolog	0.680	-2.82%
zephyr	0.703	+22.60%
ulog	0.713	+4.59%
hilog	0.827	+0.04%

表 5.8 消融实验中的字符串编码影响

分组	binary_varstr 时间改善%	binary_nocrc_varstr 时间改善%
syslog	-2.41%	+3.89%
nanolog	-6.86%	+0.38%
zephyr	+4.36%	+16.09%
ulog	-8.85%	-4.41%
hilog	+3.63%	+3.58%

如表5.7和表5.8所示，结合严格对齐结果与消融结果，可以把原因归纳为三点：1. **校验开销并非统一主导项**：本地五组中，binary_nocrc 的影响区间扩大到 -2.82% ~ +22.60%。这说明关闭校验并不会在所有语义组上自动带来收益，真正受益更明显的是 zephyr 这类紧凑定长路径。2. **冷路径差异仍然存在，但不再只集中于文本型 peer**：syslog 的 prep_ms 仍达到 8.214ms，说明系统路径初始化仍然最

重；而 nanolog/zephyr/ulog/hilog 的 prep_ms 已落在 0.680 ~ 0.827ms，说明单机主导差异更多开始转向热路径组织方式，而不是单纯的初始化成本。3. **auto-varstr 收益具有明显语义组差异**：zephyr/hilog 中 varstr 或 nocrc_varstr 仍可能带来正收益，但 nanolog/ulog/syslog 中单独开启 varstr 反而可能转负，说明变长编码并非所有 schema 的统一最优策略，必须结合字段分布来判断。

5.9 初始化竞争正确性

初始化竞争结果同步更新如下：1. 本地多进程初始化竞争。

平均耗时为 14.173 ms，$p95$ 为 16.804 ms，平均创建成功进程数为 1.0，平均复用已存在共享文件成功进程数为 9.0。

2. 真实多设备模拟（多台虚拟机节点）初始化竞争。

   全局平均耗时为 6419.830 ms，$p95$ 为 6550.884 ms，平均单创建者数量为 1.0，平均失败节点数为 0.0。

两组结果均稳定满足“单创建者、其余节点复用已初始化共享文件”的正确性目标。

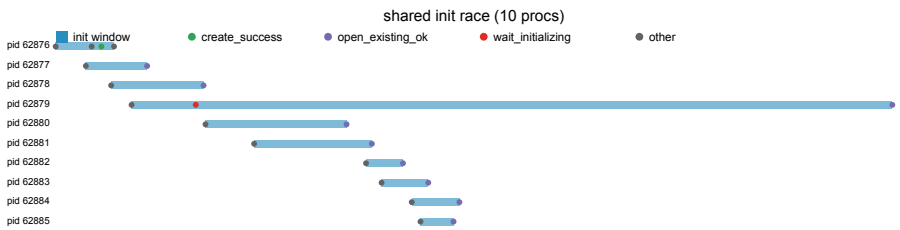

图 **5.12** 本地多进程初始化竞争时序图

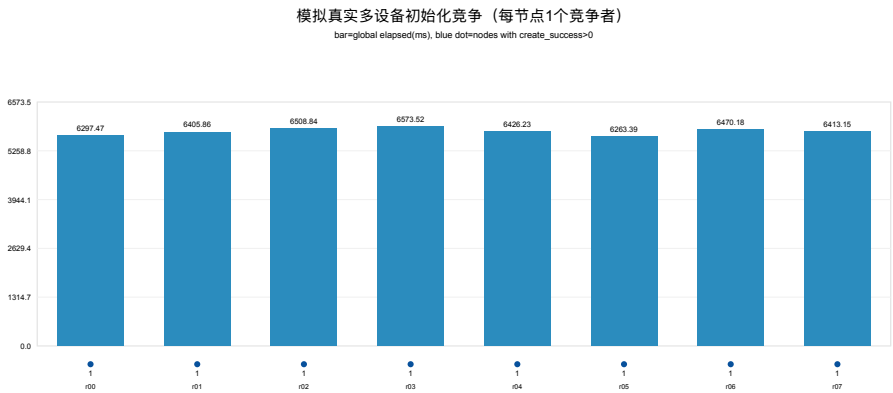

图 **5.13** 真实多设备节点级初始化竞争总览图

5.10 证据链组织与外推边界

为了让结论形成“可复核、可解释、可外推”的证据链，本文将数据集按用途组织为三类：1. **工程近似层**：保留 syslog/ftrace 以及 *_like 路径，观察真实工程风格系统的整体代价排序。2. **严格公平层**：统一使用 text_semantic_like / binary / peer native mode，在相同语义组、相同字段值和相同平台上报告单机、本地多设备与真实多设备结果。3. **原因分析层**：保留 binary_hotpath / binary_nocrc / binary_varstr 等消融模式，用于解释当前实现中的主要时间来源和空间来源。

其中，工程近似层回答“真实系统路径如何排序”，严格公平层回答“同语义条件下 Optbinlog 与 peer 如何分化”，原因分析层回答“差异主要来自哪些结构性开销”。三类数据分工明确，可避免把部署差异、语义差异与实现差异混写在同一结论中。

因此，严格公平层主结论由 nanolog/zephyr/ulog/hilog/syslog 在 single/multi/l1 三层结果与 ftrace 在 l1 结果共同支撑；其中“Optbinlog 对齐 peer 语义、peer 保持原生存储路径”是这一层结论成立的前提。工程近似层仅承担工程部署视角的补充说明。

5.11 机制解释与应用启示

结合工程近似对照、严格对齐套件和消融结果，可将 Optbinlog 的差异点归纳为三层：1. **文本扩展型语义组**：ulog/syslog 的主要矛盾仍然是文本扩展、系统路径与记录膨胀，因此 binary 在 single 与真实多设备两层场景中都保持稳定优势。这仍然是 Optbinlog 当前最明确的受益区间。2. **紧凑轻量语义组**：hilog 处在中间地带。它不像 ulog/syslog 那样带有很重的文本扩展成本，但也不像 nanolog/zephyr 那样极端紧凑，因此结果呈现为“空间长期占优、时间小幅占优、吞吐接近持平”的混合结构。3. **极短热路径语义组与 tracing 路径**：nanolog/zephyr 已经把热路径压得很短，Optbinlog 的剩余通用开销会在本地多设备与真实多设备中被进一步放大；空间扫描也显示这两组在 $N \leq 100000$ 范围内仍不存在交叉点。ftrace 则体现出另一种极端：内核 tracing 路径在空间上更紧凑，但在时间和吞吐上显著落后，说明平台路径差异会直接改变三指标的收益方向。

Optbinlog 的应用优势主要体现在：1. **跨进程一致性**：共享 schema + 偏移访问保证多进程解码一致，适合网关/边缘设备的多服务部署。2. **工程可迁移性**：不依赖编译期代码生成链路，新增日志事件只需更新 schema 文件，适合迭代频

繁的业务系统。3. **可验证性**：二进制帧包含长度与 per-record CRC32C，损坏检测与回归测试路径清晰，便于工程质量控制。4. **对文本扩展型 peer 的明显优势**：ulog/syslog 的三层对齐结果表明，binary 对文本扩展和系统路径更重的 peer 风格路径具有更清晰、更稳定的综合优势；hilog 的优势则更多体现在空间和时间，而非吞吐的全面领先。5. **优化路径清晰**：实验已经把剩余开销收敛到 cold-path + 通用打包路径 + 稳态缓存收益释放不足，后续优化可以围绕长生命周期进程、steady-state fast path 和批量提交策略展开。

完整分析表、最新可视化和关键结论汇总见严格对齐套件生成的报告；工程近似层的补充说明见实验补充分析报告。

---

6 结论与展望

6.1 研究结论

本文形成了“调研 → 设计 → 建模 → 验证 → 解释”的完整研究链条：

1. 调研层：识别了代表系统的共性优化机制（deferred、ring buffer、结构化二进制、热路径迁移）及其适用边界。
2. 设计层：Optbinlog 通过共享 schema 与偏移访问实现跨进程一致性，并以自描述帧与硬件 CRC32C 保持可校验性。
3. 理论层：建立了 N、D 与时间/空间/吞吐之间的关系模型，给出可检验的交叉点与饱和点预测。
4. 实验层：在 single、本地多设备与真实多设备模拟三层场景完成严格语义对齐验证，并通过初始化竞争和消融实验解释差异来源。

从结果上看，binary 相对 text 在本轮纳入的各语义组上均为正收益；在对齐到 peer 语义并保留 peer 原生存储路径后，ulog/syslog 在真实多设备场景中保持稳定综合优势，hilog 呈现“空间占优、时间略优、吞吐近持平”的中间形态，nanolog/zephyr 仍未越过节点级交叉点，ftrace 则在真实多设备中表现为“空间正收益、时间吞吐负收益”。空间扫描显示 ulog/syslog 在 $N \approx 10$ 即出现反超，hilog 在 $N \approx 50$ 左右出现反超，而 nanolog/zephyr 在 $N \leq 100000$ 仍未出现空间交叉。上述结论与第 4 章模型保持一致，说明 Optbinlog 更适合“长期在线、版本持续演进、强调跨进程一致性”的嵌入式网关与边缘控制器场景。

6.2 当前局限

1. 受网络与平台限制，尚未完成外部系统官方工程的全栈编译复现（当前为代码级阅读 + 算法级原型复现）。
2. Linux 实验仍以 VM 为主，尚未覆盖 Raspberry Pi / i.MX 实机。
3. 严格公平语义套件已经扩展到 20 节点 VM 级 “真实多设备” 模拟，但仍未迁移到真实 ARM 板卡或 bare-metal Linux 实机。
4. 当前严格公平套件采用 “新进程 + 单次写入” 统计口径，因此新加入的进程内 shared/tag cache 收益只在代码层完成接入，尚未通过长生命周期 steady-state 基准充分量化。
5. 缺少 perf 级 CPU 微观指标，暂不能细粒度解释所有异常点。

6.3 未来展望

未来工作可从以下方向持续推进：1. 在更广泛的平台与负载条件下开展验证，进一步提升结论的外部有效性与可迁移性。2. 在保持语义一致性的前提下继续优化热路径与稳态路径，缩小与极短路径 peer 在特定场景下的差距。3. 完善日志可靠性与可恢复性评估，形成覆盖损坏、截断与异常恢复过程的系统化证据。4. 结合更细粒度观测手段（如 CPU 与调度行为分析），建立从机制到性能现象的更强解释闭环。

参考文献

[1] Linux Kernel Documentation. Lockless Ring Buffer Design[EB/OL]. https://www.kernel.org/doc/html/latest/trace/ring-buffer-design.html.

[2] Linux Kernel Documentation. user_events[EB/OL]. https://www.kernel.org/doc/html/latest/trace/user_events.html.

[3] LTTng Project. LTTng Documentation: Concepts[EB/OL]. https://lttng.org/docs/#doc-concepts.

[4] Zephyr Project. Logging[EB/OL]. https://docs.zephyrproject.org/latest/services/logging/index.html.

[5] Zephyr Project. Kconfig.mode[EB/OL]. https://pigweed.googlesource.com/third_party/github/zephyrproject-rtos/zephyr/+/refs/heads/upstream/collab-sdk-dev/subsys/logging/Kconfig.mode.

[6] Zephyr Project. log_msg.c[EB/OL]. https://pigweed.googlesource.com/third_party/github/zephyrproject-rtos/zephyr/+/refs/heads/main/subsys/logging/log_msg.c.

[7] systemd. Journal File Format[EB/OL]. https://systemd.io/JOURNAL_FILE_FORMAT/.

[8] Yang S, Park W, Byan T, et al. NanoLog: A Nanosecond Scale Logging System[C]. USENIX Annual Technical Conference, 2018.

[9] PlatformLab. NanoLogCpp17.h[EB/OL]. https://raw.githubusercontent.com/PlatformLab/NanoLog/master/runtime/NanoLogCpp17.h.

[10] PlatformLab. RuntimeLogger.cc[EB/OL]. https://raw.githubusercontent.com/PlatformLab/NanoLog/master/runtime/RuntimeLogger.cc.

[11] Android Open Source Project. event.logtags[EB/OL]. https://android.googlesource.com/platform/system/logging/+/refs/tags/android-13.0.0_r33/logcat/event.logtags.

[12] RT-Thread. ulog 文档 [EB/OL]. https://rt-thread.github.io/rt-thread/page_component_ulog.html.

[13] RT-Thread. ulog.c[EB/OL]. https://raw.githubusercontent.com/RT-Thread/rt-thread/master/components/utilities/ulog/ulog.c.

[14] OpenHarmony. HiLog Lite README[EB/OL]. https://gitee.com/openharmony/hiviewdfx_hilog_lite/blob/master/README_zh.md.

[15] Linux Kernel. ring_buffer.c[EB/OL]. https://raw.githubusercontent.com/torvalds/linux/master/kernel/trace/ring_buffer.c.

[16] systemd. journald-server.c[EB/OL]. https://github.com/systemd/systemd/blob/main/src/journal/journald-server.c.

[17] Sigelman B H, Barroso L A, Burrows M, et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure[C]. Google Research, 2010.

[18] Cantrill B M, Shapiro M W, Leventhal A H. Dynamic Instrumentation of Production Systems[C]. USENIX Annual Technical Conference, 2004.

[19] Zhu J, He P, Fu Q, et al. Learning to Log: Helping Developers Make Informed Logging Decisions[C]. ICSE, 2015.

[20] OpenTelemetry. Logs Data Model[EB/OL]. https://opentelemetry.io/docs/specs/otel/logs/data-model/.

致谢

本论文（设计）完成过程中，得到了指导教师在选题、方法、实验组织和写作规范方面的持续指导；也得到了课题组同学在实验环境、脚本复核与结果讨论方面的帮助。在此一并致以诚挚感谢。感谢学院和学校提供的学习与实验条件，感谢家人长期以来的理解与支持。

6.4 附录 A 关键复现实验命令（阶段化复现）

```
# 1) 全分组语义对齐矩阵（single/multi/真实多设备模拟）
cd demo
python3 run_grouped_semantic_matrix.py \
```

```
  --out-dir "$(pwd)/results/paper_regen/grouped_semantic_matrix"

# 2) 最终严格对齐套件（single/本地多设备/真实多设备 5,10,15,20 节点）
python3 run_final_aligned_suite.py \
  --out-dir "$(pwd)/results/paper_regen/final_aligned_suite"

# 3) 本地初始化竞争（10 进程）
clang -O2 -Wall -Wextra -std=c11 -Iinclude \
  -o optbinlog_init_race optbinlog_init_race.c \
  src/optbinlog_shared.c src/optbinlog_eventlog.c
OPTBINLOG_INIT_OUT_DIR="$(pwd)/results/paper_regen/init_race_local" \
OPTBINLOG_EVENTLOG_DIR="$(pwd)/eventlogst" \
OPTBINLOG_INIT_PROCS=10 OPTBINLOG_INIT_REPEATS=20 OPTBINLOG_INIT_WARMUP=3 \
python3 run_init_race.py

# 4) 真实多设备模拟节点级初始化竞争（每节点一台设备）
python3 run_l1_init_compete.py \
  --config l1_config.linux_10_all_unaligned_initrace.json \
  --tag paper_init_compete

# 5) Optbinlog 便捷读取（字段名 + 统计）
clang -O2 -Wall -Wextra -std=c11 -Iinclude \
  -o optbinlog_read optbinlog_read.c \
  src/optbinlog_shared.c src/optbinlog_eventlog.c src/optbinlog_binlog.c
./optbinlog_read \
  --shared ./results/paper_dataset/samples/optbinlog_read/shared_eventtag.bin \
  --log ./results/paper_dataset/samples/optbinlog_read/binary_run_000.bin \
  --format table --limit 20 --summary
```